

Container With Most Water

Proof of Correctness — Two-Pointer Greedy Approach

1. Goal

To prove that the greedy, two-pointer approach is correct. Specifically, we want to prove that at each step, the optimal strategy is to move the pointer corresponding to the **shorter** of the two vertical lines.

2. Setup

- We use two pointers, **left** and **right**, initialized at the two ends of the **height** array.
- The area A formed by these two lines is calculated as shown below.

$$A = \text{width} \times \text{height}$$

The width is the distance between the pointers:

$$\text{width} = \text{right} - \text{left}$$

The height is limited by the shorter of the two lines:

$$\text{height} = \min(\text{height}[\text{left}], \text{height}[\text{right}])$$

3. The Proof by Case Analysis

Let's analyze the state at any given step. To simplify, we'll assume without loss of generality that **height[left] < height[right]**. The argument is symmetrical if the opposite is true.

The current area, A_{current} , is limited by the shorter line, **height[left]**:

$$A_{\text{current}} = (\text{right} - \text{left}) \times \text{height}[\text{left}]$$

We now have two possible moves:

- Increment the **left** pointer ($\text{left} \rightarrow \text{left} + 1$).
- Decrement the **right** pointer ($\text{right} \rightarrow \text{right} - 1$).

Let's evaluate the potential outcome of each move.

Case A — Move the Taller Line's Pointer (Decrement right)

If we move the pointer of the taller line (right), the new state will be:

- **New Width:** $\text{width}' = (\text{right} - 1) - \text{left}$. This is **strictly smaller** than the original width: $\text{width}' < (\text{right} - \text{left})$.

- **New Height:** $\text{height}' = \min(\text{height}[\text{left}], \text{height}[\text{right} - 1])$. Since $\text{height}[\text{left}]$ was the original limiting height, the new height can at best be **equal** to it: $\text{height}' \leq \text{height}[\text{left}]$.

The new area, A_{new} , is the product of a strictly smaller width and a height that is less than or equal to the original. This means the new area is guaranteed to be smaller than the current area:

$$A_{\text{new}} = \text{width}' (< \text{original}) \times \text{height}' (\leq \text{original}) \Rightarrow A_{\text{new}} < A_{\text{current}}$$

Conclusion for Case A: Moving the pointer of the taller line can never produce a larger area. We lose nothing by discarding this option.

Case B — Move the Shorter Line's Pointer (Increment left)

If we move the pointer of the shorter line (left), the new state will be:

- **New Width:** $\text{width}'' = \text{right} - (\text{left} + 1)$. The width still decreases.
- **New Height:** $\text{height}'' = \min(\text{height}[\text{left} + 1], \text{height}[\text{right}])$.

Unlike in Case A, the new limiting height, height'' , is **not** constrained by our original $\text{height}[\text{left}]$. It is possible that $\text{height}[\text{left} + 1]$ is significantly taller than $\text{height}[\text{left}]$. If $\text{height}[\text{left} + 1]$ is large enough, this increase in height could potentially overcome the decrease in width, leading to a larger overall area:

$$\text{It is possible that } A_{\text{new}} > A_{\text{current}}$$

4. Final Conclusion

Given the state where $\text{height}[\text{left}] < \text{height}[\text{right}]$:

- Moving **right** (the taller line) guarantees a **smaller** area.
- Moving **left** (the shorter line) is the only choice that holds the potential to find a **larger** area.

Therefore, the optimal greedy strategy is to always eliminate the shorter line and move its pointer inward in search of a better container. This proves the correctness of the two-pointer algorithm.

Complexity

- **Time complexity:** $O(n)$ — each pointer moves at most n times total.
- **Space complexity:** $O(1)$ — only a constant number of variables are used.